PARADIGMAS DE PROGRAMAÇÃO
PROGRAMAÇÃO PROCEDURAL (MODULARIZAÇÃO)

Linguagem padrão: pseudocódigo (Portugol).

Objetivo: ensinar programação procedural (funções/procedimentos), com exemplos e exercícios, para consolidar organização e manutenção de algoritmos.

1. O que é Programação Procedural

Programação procedural é um estilo de programação imperativo em que o problema é resolvido por meio de PROCEDIMENTOS e FUNÇÕES (módulos).

A ideia central é simples: em vez de colocar tudo em um algoritmo principal grande, você divide o algoritmo em partes pequenas, cada uma com uma responsabilidade clara.

- Reduz repetição: uma regra fica em um lugar.
- Facilita leitura: o programa vira uma sequência de chamadas com nomes claros.
- Facilita teste: você consegue testar uma função isoladamente.
- Facilita mudança: quando o requisito muda, você altera o módulo responsável.

2. Procedimento vs Função

Procedimento: executa uma ação, normalmente sem retornar um valor (ex.: imprimir um relatório).

Função: recebe entradas (parâmetros) e retorna um valor (ex.: calcular subtotal).

- Procedimento = 'faz algo'.
- Função = 'calcula e devolve algo'.

3. Componentes básicos de um módulo

- Nome claro: diz o que faz (ex.: calcularFrete).
- Parâmetros: dados que entram.
- Variáveis locais: existem só dentro do módulo.
- Retorno (em função): valor devolvido.
- Evite efeitos colaterais desnecessários: prefira passar dados por parâmetro a usar variável global.

4. Boas práticas (nível iniciante)

- Uma função deve fazer uma coisa só (uma regra principal).
- Evite funções longas (se ficou grande, divida).
- Prefira passar dados por parâmetro a usar variável global.
- Nomeie variáveis com significado (subtotal, desconto, frete).
- Trate casos inválidos no começo (ex.: N=0, cupom desconhecido).

5. Exemplo 1 — Função simples (par/ímpar)

Objetivo: criar uma função que retorna VERDADEIRO se um número for par.

```
  funcao logico ehPar(inteiro num) {
    retorne num % 2 == 0
  }
```

Uso:

```
  funcao inicio() {
    inteiro numero
    escreva("Informe um número: ")
    leia(numero)

    se (ehPar(numero)) {
      escreva("É par")
    } senao {
      escreva("É ímpar")
    }
  }
```

6. Exemplo 2 — Procedimento para imprimir relatório

```
  funcao vazio imprimirResumo(real subtotal, real desconto, real frete,
real taxa, real total) {
    escreva("Subtotal: ", subtotal, "\n")
    escreva("Desconto: ", desconto, "\n")
    escreva("Frete: ", frete, "\n")
    escreva("Taxa: ", taxa, "\n")
    escreva("Total: ", total, "\n")
  }
```

7. Exemplo 3 (completo) — Mini-sistema de Caixa (procedural)

Neste exemplo, o programa lê itens, calcula subtotal, aplica cupom, calcula frete e taxa de pagamento e imprime o total.

Regras:

- Cupom 'DESC10': 10% sempre.
- Cupom 'DESC20': 20% apenas se subtotal >= 150.
- Frete: 15; se subtotal >= 200, frete = 0.
- Pagamento: 'PARCELADO' aplica 5% sobre (subtotal - desconto + frete).

Estrutura recomendada (módulos):

- calcularSubtotal(precos[]) -> real
- calcularDesconto(subtotal, cupom) -> real
- calcularFrete(subtotal) -> real
- calcularTaxaPagamento(valorBase, tipoPagamento) -> real
- calcularTotal(subtotal, desconto, frete, taxa) -> real
- imprimirResumo(...) (procedimento)

Pseudocódigo completo (modelo):

```
programa
{
    // Ajuste o tamanho máximo conforme sua turma/atividade
    const inteiro MAX_ITENS = 100

    funcao real calcularSubtotal(real precos[], inteiro n)
    {
        inteiro i
        real soma = 0.0

        para (i = 0; i < n; i++)
        {
            soma = soma + precos[i]
        }

        retorne soma
    }

    funcao real calcularDesconto(real subtotal, cadeia cupom)
    {
        se (cupom == "DESC10")
        {
```

```
            retorne subtotal * 0.10
        }
        senao se ((cupom == "DESC20") e (subtotal >= 150))
        {
            retorne subtotal * 0.20
        }
        senao
        {
            retorne 0.0
        }
    }

    funcao real calcularFrete(real subtotal)
    {
        se (subtotal >= 200)
        {
            retorne 0.0
        }
        senao
        {
            retorne 15.0
        }
    }

    funcao real calcularTaxaPagamento(real valorBase, cadeia
tipoPagamento)
    {
        se (tipoPagamento == "PARCELADO")
        {
            retorne valorBase * 0.05
        }
        senao
        {
            retorne 0.0
        }
    }

    funcao real calcularTotal(real subtotal, real desconto, real frete,
real taxa)
    {
        retorne (subtotal + frete + taxa - desconto)
    }

    funcao vazio imprimirResumo(real subtotal, real desconto, real frete,
real taxa, real total)
```

```
{
    escreva("Subtotal: ", subtotal, "\n")
    escreva("Desconto: ", desconto, "\n")
    escreva("Frete: ", frete, "\n")
    escreva("Taxa: ", taxa, "\n")
    escreva("Total: ", total, "\n")
}

funcao inicio()
{
    inteiro n, i
    real precos[MAX_ITENS]
    cadeia cupom, tipoPagamento
    real subtotal, desconto, frete, valorBase, taxa, total

    escreva("Quantidade de itens (max ", MAX_ITENS, "): ")
    leia(n)

    se (n <= 0 ou n > MAX_ITENS)
    {
        escreva("Quantidade invalida.\n")
        retorne
    }

    para (i = 0; i < n; i++)
    {
        escreva("Preco do item ", (i + 1), ": ")
        leia(precos[i])

        se (precos[i] < 0)
        {
            escreva("Preco invalido.\n")
            retorne
        }
    }

    escreva("Cupom (DESC10, DESC20 ou outro): ")
    leia(cupom)

    escreva("Tipo pagamento (AVISTA ou PARCELADO): ")
    leia(tipoPagamento)

    subtotal = calcularSubtotal(precos, n)
    desconto = calcularDesconto(subtotal, cupom)
    frete = calcularFrete(subtotal)
```

```
        valorBase = subtotal - desconto + frete
        taxa = calcularTaxaPagamento(valorBase, tipoPagamento)

        total = calcularTotal(subtotal, desconto, frete, taxa)

        imprimirResumo(subtotal, desconto, frete, taxa, total)
    }
}
```

8. Como testar um programa procedural (teste por módulo)

Em programação procedural, você pode testar cada função isoladamente com entradas conhecidas e comparar a saída com a esperada.

Exemplo de testes (calcularDesconto):

- Entrada: subtotal=100, cupom='DESC10' → saída esperada: 10
- Entrada: subtotal=200, cupom='DESC20' → saída esperada: 40
- Entrada: subtotal=120, cupom='DESC20' → saída esperada: 0

EXERCÍCIOS (Procedural)

A) Aquecimento — funções pequenas

1. Crie a função max2(a,b) que retorna o maior entre dois inteiros.
2. Crie a função min3(a,b,c) que retorna o menor entre três inteiros.
3. Crie a função ehMultiplo(a,b) que retorna VERDADEIRO se a é múltiplo de b (b != 0).
4. Crie a função absoluto(x) que retorna |x|.
5. Crie a função media2(n1,n2) que retorna a média de dois reais.

B) Decisão — funções com regras

6. Crie a função classificarIdade(idade) que retorna 'CRIANCA', 'ADOLESCENTE', 'ADULTO' ou 'IDOSO'.
7. Crie a função calcularFrete(valor) com regra: frete=0 se valor>=200; senão 15.
8. Crie a função calcularDesconto(subtotal,cupom) (DESC10 e DESC20 com subtotal>=150).
9. Crie a função podeEntrar(idade, autorizado) que retorna VERDADEIRO se idade>=18 OU autorizado=verdadeiro.

C) Refatoração — do monolítico para procedural

Entrega: pseudocódigo final + nomes das funções + 3 casos de teste (entradas/saídas esperadas).

10. Refatore: ler N notas (0..10), validar entradas, calcular média e classificar (>=7 aprovado; >=5 recuperação; senão reprovado). Funções sugeridas: validarNota, calcularMedia, classificarSituacao.
11. Refatore: ler N números e imprimir maior, menor e média. Funções: atualizarMaiorMenor, calcularMedia.
12. Refatore: triagem (temperatura, faltaAr, idade) e imprimir risco alto/baixo. Funções: ehRiscoAlto, imprimirResultado.

D) Mini-projeto — Caixa Procedural

- Implemente o mini-sistema de Caixa (modelo da apostila), usando no mínimo 5 funções/procedimentos.
- Entrega: pseudocódigo + 4 casos de teste (2 com cupom e 2 sem).
- Obrigatório: 1 teste de mesa completo de uma função (ex.: calcularDesconto).